PYQ Bank: Previous-Year Question Bank with Repeat Detection

DBMS Experiential Learning, Level 3 Project · Department of Computer Science and Engineering, PES University

Name	SRN
Swaroop Venkateshwar	PES1UG24CS923
Shreesh S A	PES1UG24CS441

Database option: **Option 2: SQL + Vector Database** (PostgreSQL + the pgvector extension).

---

1. Objectives

Students prepare for exams with previous-year question papers (PYQs), but the papers are scattered PDFs, and nobody can easily tell which questions *keep coming back*. Questions are rarely repeated word for word. "Explain the ACID properties with a bank-transfer example" comes back next year as "Define a transaction and explain atomicity, consistency, isolation and durability using a money transfer". A keyword search cannot see that these are the same question.

PYQ Bank aims to:

1. Store every exam paper (ISA1, ISA2, ESA) of every course in a well-designed relational database, with each question tagged by unit and topic.
2. **Detect repeated questions automatically**, including reworded ones, using vector embeddings and similarity search inside the database.
3. Let faculty confirm or reject uncertain matches, so the database learns from human decisions.
4. Give students useful views: the most repeated questions, how marks are spread across units, how often each topic is asked, and search by keyword *or* by meaning.
5. Demonstrate the DBMS concepts of the course on a real problem: design, normalisation, constraints, triggers, procedures, views, transactions, locking, indexing, full-text search and vector search.

2. Problem interpretation: what counts as a "repeat"

The project handles two kinds of repetition:

Kind	Example	How it is detected
Same question, any wording	"Explain 2PL. Does it guarantee serializability?" vs "Describe the two-phase locking protocol and why it ensures conflict serializability"	Cosine similarity of sentence embeddings (pgvector), grouped into a **repeat group**
Same topic asked again	Two *different* questions on "Concurrency control and locking"	Every question is tagged with a topic, so topic frequency comes from `GROUP BY` queries (`v_topic_frequency`)

3. Functional requirements

Student - Register with SRN and log in. - Browse courses; for each course see the *most repeated questions* ranked by likelihood, *unit weightage*, *topic frequency* and *past papers*. - Open a paper and see, next to each question, whether it was asked before and where. - Open a repeat group to see every wording of a question across the years, with similarity scores. - Search questions by keyword (full-text) and by meaning (vector), side by side, filtered by course and exam type. - Bookmark questions into a personal practice list.

Faculty - See the courses they are assigned to. - Create a draft paper, add questions one by one. Each question is embedded and checked against all earlier papers of the course *immediately*: grouped as a repeat, sent for review, or marked new. - Publish a paper (after which its questions are locked). - Review the queue of uncertain pairs: mark them as the same question or as different. - Remove a wrongly grouped question from a repeat group.

Admin (exam cell) - Dashboard: papers, questions, repeat groups, repeat share per year, review backlog, recent changes. - Manage courses, units and topics; create faculty accounts; assign faculty to courses; enable or disable accounts. - Tune the detector's similarity thresholds. - SQL Lab: run the project's important queries live (read-only). - Audit log: every change to papers, questions, groups, suggestions and settings, with the user who made it.

Non-functional - Credentials only in environment variables (`.env` , git-ignored). - Passwords stored as bcrypt hashes; sessions as signed, http-only JWT cookies. - All validation that protects data integrity is enforced **in the database** (constraints and triggers), not only in the UI.

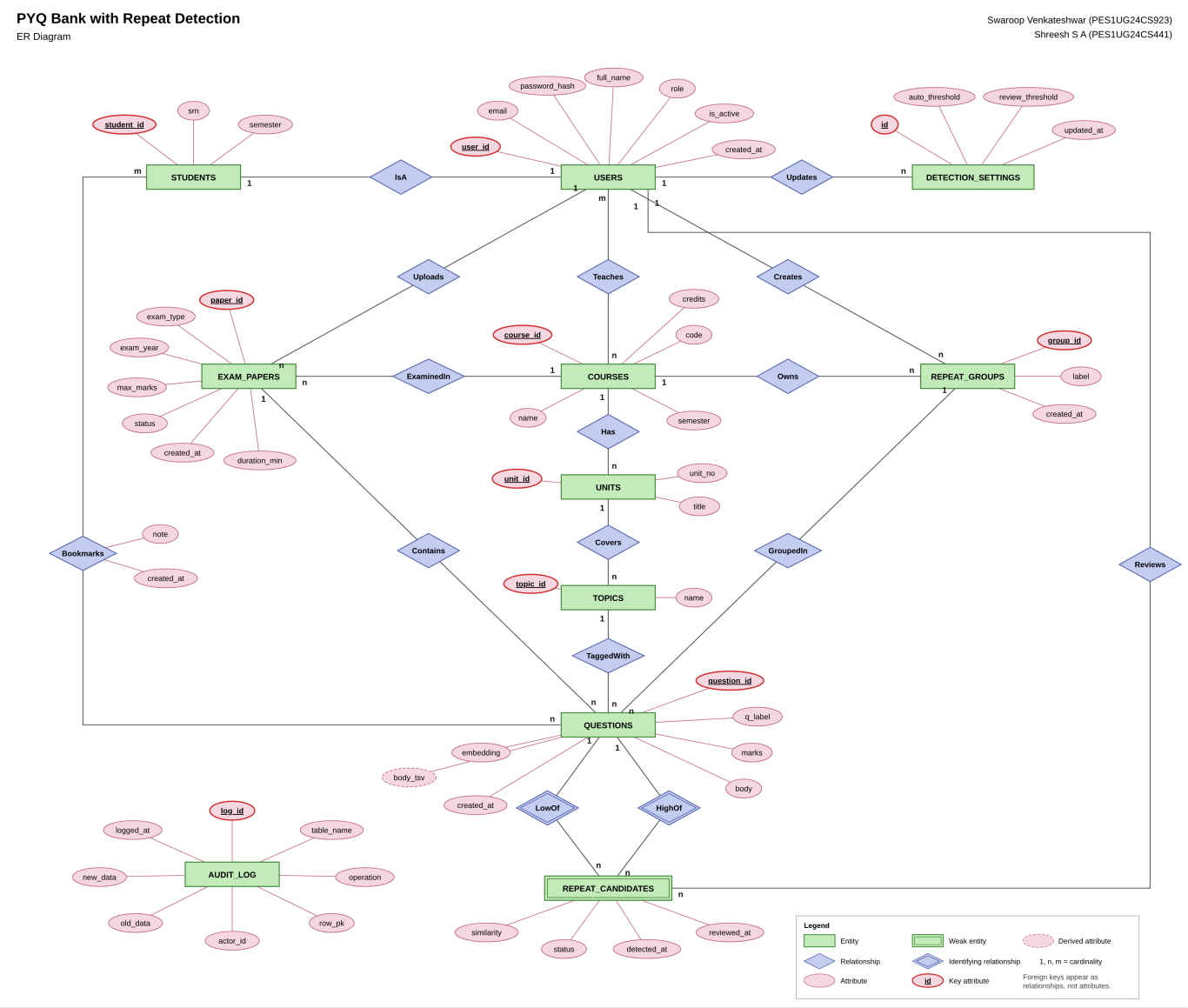
4. Database choice and justification

Data	Stored in	Why
Users, students, courses, units, topics, faculty assignments, exam papers, questions, repeat groups, review decisions, bookmarks	**PostgreSQL tables**	The data is highly structured and full of relationships (1:1, 1:N, M:N) and rules (a question's topic must belong to the paper's course; a published paper is locked; a pair can be reviewed once). This needs keys, foreign keys, CHECK constraints, triggers, joins, aggregates and ACID transactions, which is exactly what a relational DBMS provides.
Meaning of each question	**pgvector** column `questions.embedding vector(384)` with an HNSW index	"Is this the same question in other words?" is a *similarity* question, not an equality question. Keyword matching fails on rewordings; embeddings place questions with the same meaning close together, and a vector index finds the nearest ones quickly.
Words of each question	`questions.body_tsv tsvector` (generated) with a GIN index	For exact-term search ("deadlock", "BCNF") full-text search is precise and cheap, and it complements meaning search.

Why pgvector instead of a separate vector database (Qdrant, Pinecone, Chroma)? Every vector belongs to exactly one question row. Keeping it in the same row means: - the vector and the relational data are always consistent (one transaction inserts both); - the repeat detector can run *inside a trigger*, combining vector search with relational filters (same course, different paper) in one SQL statement; - there is no second system to keep in sync.

The handout lists "PostgreSQL with pgvector" as a valid vector database for Option 2. A NoSQL document store was not chosen because nothing in this domain is schema-less: every question has the same structure and many relationships.

5. ER diagram



Chen notation: entities are rectangles (REPEAT_CANDIDATES is a weak entity, identified through the two identifying relationships LowOf and HighOf), relationships are diamonds, attributes are ovals (key attributes underlined, `body_tsv` dashed because it is derived). Foreign keys appear as relationships, not attributes. Printable copy: `docs/diagrams/ER_Diagram_PYQBank.pdf`.

Entities and relationships:

Relationship	Cardinality	Notes
USERS – STUDENTS	1 : 0..1	Specialisation: a student *is a* user (shared primary key)
USERS – COURSES via FACULTY_COURSES	M : N	Which faculty may upload/review for which course
COURSES – UNITS – TOPICS	1 : N, 1 : N	Syllabus hierarchy
COURSES – EXAM_PAPERS	1 : N	One paper per (course, exam type, year)

Relationship	Cardinality	Notes
EXAM_PAPERS – QUESTIONS	1 : N	Identifying: a question is numbered within its paper (`UNIQUE (paper_id, q_label)`)
TOPICS – QUESTIONS	1 : N	Every question is tagged with exactly one topic
REPEAT_GROUPS – QUESTIONS	0..1 : N	A question belongs to at most one repeat group
QUESTIONS – QUESTIONS via REPEAT_CANDIDATES	M : N (recursive)	Suggested pairs, stored once with `q_low < q_high`
STUDENTS – QUESTIONS via BOOKMARKS	M : N	Practice list, with an optional note

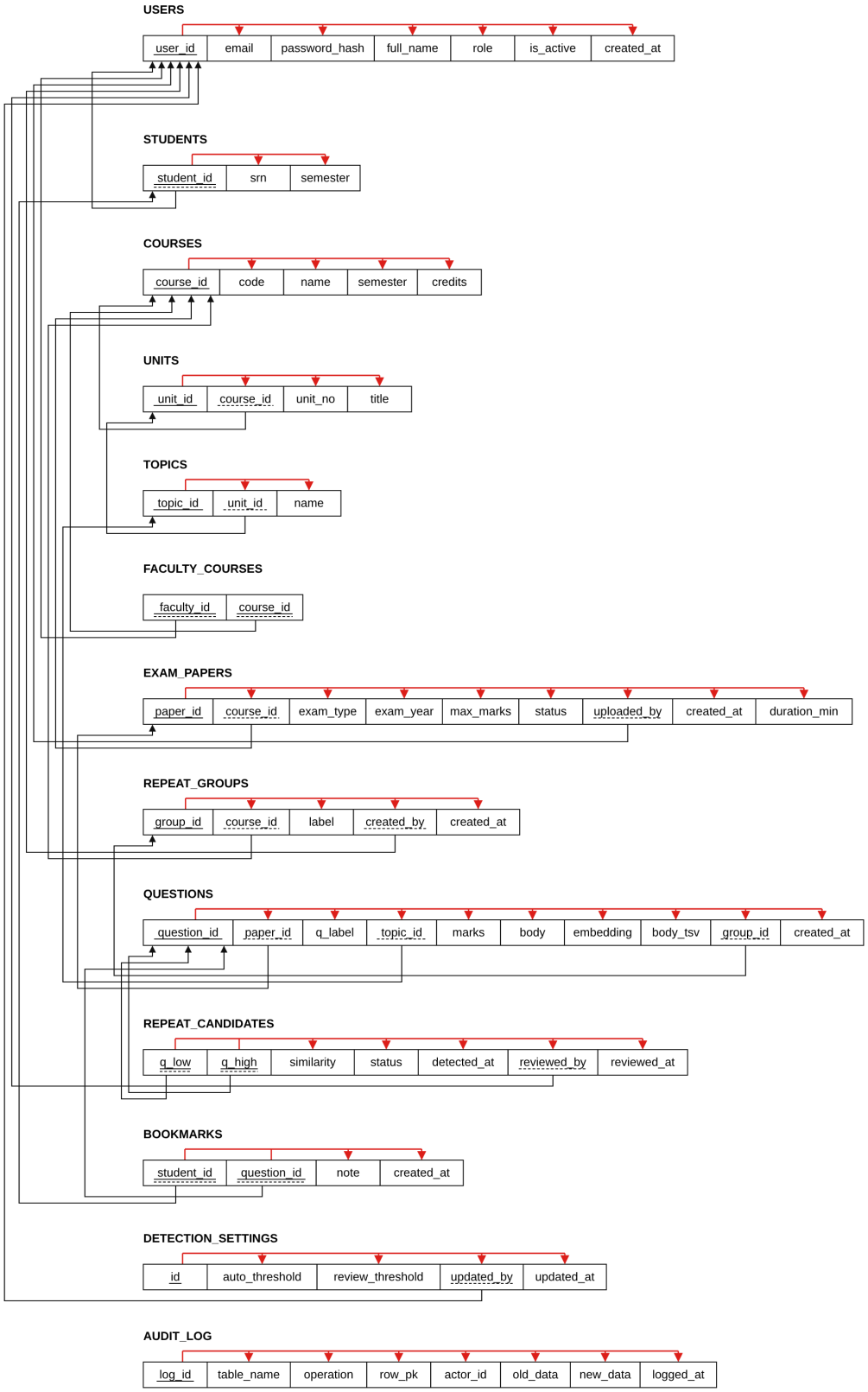
6. Relational schema

PYQ Bank with Repeat Detection

11 November 2026

Relational Schema (BCNF)

Swaroop Venkateshwar (PES1UG24CS923) · Shreesh S A (PES1UG24CS441)



Underline = primary key · dashed underline = foreign key · red arrows = functional dependencies (key → attribute) · black arrows = foreign key → referenced primary key

Underline = primary key, dashed underline = foreign key, red arrows = functional dependencies (key → attribute), black arrows = foreign key → referenced primary key. Printable copy: `docs/diagrams/Schema_Diagram_PYQBank.pdf`.

In text form (primary keys in **bold**, foreign keys marked →):

- users(**user_id**, email UNIQUE, password_hash, full_name, role, is_active, created_at)
- students(**student_id** → users, srn UNIQUE, semester)
- courses(**course_id**, code UNIQUE, name, semester, credits)
- units(**unit_id**, course_id → courses, unit_no, title) · UNIQUE(course_id, unit_no)
- topics(**topic_id**, unit_id → units, name) · UNIQUE(unit_id, name)
- faculty_courses(**faculty_id** → users, **course_id** → courses)
- exam_papers(**paper_id**, course_id → courses, exam_type, exam_year, max_marks, duration_min, status, uploaded_by → users, created_at) · UNIQUE(course_id, exam_type, exam_year)
- repeat_groups(**group_id**, course_id → courses, label, created_by → users, created_at)
- questions(**question_id**, paper_id → exam_papers, q_label, topic_id → topics, marks, body, embedding, body_tsv, group_id → repeat_groups, created_at) · UNIQUE(paper_id, q_label)
- repeat_candidates(**q_low** → questions, **q_high** → questions, similarity, status, detected_at, reviewed_by → users, reviewed_at)
- bookmarks(**student_id** → students, **question_id** → questions, note, created_at)
- detection_settings(**id**, auto_threshold, review_threshold, updated_by → users, updated_at)
- audit_log(**log_id**, table_name, operation, row_pk, actor_id, old_data, new_data, logged_at)

7. Normalisation (functional dependencies and normal forms)

1NF. Every attribute is atomic. Multi-valued facts are separate relations: a faculty member's courses (`faculty_courses`), a student's bookmarks (`bookmarks`), the units of a course (`units`). There are no repeating groups such as `topic1, topic2, …` columns. (`embedding` is a single value of the `vector` type, compared as a whole; `body_tsv` is a single generated value.)

Functional dependencies (non-trivial, minimal):

Relation	FDs
users	user_id → email, password_hash, full_name, role, is_active; email → user_id
students	student_id → srn, semester; srn → student_id
courses	course_id → code, name, semester, credits; code → course_id
units	unit_id → course_id, unit_no, title; (course_id, unit_no) → unit_id
topics	topic_id → unit_id, name; (unit_id, name) → topic_id
exam_papers	paper_id → course_id, exam_type, exam_year, max_marks, duration_min, status, uploaded_by; (course_id, exam_type, exam_year) → paper_id
questions	question_id → paper_id, q_label, topic_id, marks, body, group_id; (paper_id, q_label) → question_id; body → body_tsv
repeat_candidates	(q_low, q_high) → similarity, status, reviewed_by, reviewed_at

2NF. In every relation with a composite key (`faculty_courses`, `repeat_candidates`, `bookmarks`), no non-key attribute depends on only *part* of the key. For example, `similarity` depends on the *pair* (q_low, q_high), not on either question alone. All other relations have single-attribute keys, so 2NF holds trivially.

3NF / BCNF. For every FD X → A above, X is a (super)key: the alternate keys (`email`, `srn`, `code`, `(course_id, unit_no)`, `(unit_id, name)`, `(course_id, exam_type, exam_year)`, `(paper_id, q_label)`) are candidate keys, so FDs from them do not violate BCNF.

Two design decisions keep the schema in BCNF that a first draft would have broken:

1. **A question stores `topic_id` only, not `unit_id` or `course_id`.** Since topic_id → unit_id → course_id, storing `unit_id` in `questions` would create a transitive dependency question_id → topic_id → unit_id (a 3NF violation and an update anomaly if a topic were moved). The unit and course are obtained by joins (see `v_question_detail`). The business rule "the topic must belong to the paper's course" is enforced by a trigger instead of by redundancy.
2. **Repeat groups are a separate relation** rather than a "repeat_of question_id" column chain, so membership is one FD (question_id → group_id) and merging two groups is a single UPDATE.

`body → body_tsv` is technically a dependency on a non-key attribute. It is a **generated column** (`GENERATED ALWAYS AS … STORED`): PostgreSQL computes it itself, so it can never be inconsistent with `body`, and storing it is what allows the GIN index. Similarly, `embedding` is derived from `body` by a model outside the database; the trigger `trg_question_integrity` clears it whenever `body` changes, so a stale vector can never remain.

8. Data dictionary

questions

Column	Type	Constraints	Meaning
question_id	SERIAL	PK	
paper_id	INT	NOT NULL, FK → exam_papers ON DELETE CASCADE	paper containing the question
q_label	VARCHAR(4)	NOT NULL, CHECK `^[1-9][0-9]?[a-e]?$`, UNIQUE with paper_id	question number, e.g. 2b
topic_id	INT	NOT NULL, FK → topics ON DELETE RESTRICT	syllabus topic
marks	NUMERIC(4,1)	NOT NULL, CHECK 0 < marks ≤ 20	
body	TEXT	NOT NULL, CHECK length ≥ 10	question text
embedding	vector(384)	HNSW index	meaning of the text (all-MiniLM-L6-v2)
body_tsv	tsvector	GENERATED from body, GIN index	words of the text, for full-text search
group_id	INT	FK → repeat_groups ON DELETE SET NULL	repeat group, NULL if not a repeat

exam_papers

Column	Type	Constraints	Meaning
paper_id	SERIAL	PK	
course_id	INT	NOT NULL, FK → courses ON DELETE RESTRICT	
exam_type	ENUM exam_type	ISA1 / ISA2 / ESA	
exam_year	SMALLINT	CHECK 2015–2035	

Column	Type	Constraints	Meaning
max_marks	SMALLINT	CHECK 10–100	
duration_min	SMALLINT	CHECK 30–240 (added by ALTER TABLE)	
status	ENUM paper_status	draft / published, DEFAULT draft	published papers are locked
uploaded_by	INT	FK → users ON DELETE SET NULL	
		UNIQUE (course_id, exam_type, exam_year)	one paper per exam

repeat_candidates

Column	Type	Constraints	Meaning
q_low, q_high	INT	PK (q_low, q_high), FKs → questions, CHECK q_low < q_high	the pair, stored once
similarity	NUMERIC(5,4)	CHECK 0–1	cosine similarity when detected
status	ENUM candidate_status	pending / confirmed / rejected	
reviewed_by, reviewed_at	INT, TIMESTAMPTZ	CHECK (status = 'pending') = (reviewed_at IS NULL)	who decided, and when

users / students

Column	Type	Constraints
users.email	VARCHAR(120)	UNIQUE, CHECK email format, CHECK lower-case
users.role	ENUM user_role	admin / faculty / student
users.password_hash	VARCHAR(100)	bcrypt hash (never the password)
students.student_id	INT	PK, FK → users ON DELETE CASCADE
students.srn	VARCHAR(13)	UNIQUE, CHECK `^PES[12]UG[0-9]{2}[A-Z]{2}[0-9]{3}$`

Other tables

- **courses**: code UNIQUE with CHECK `^UE[0-9]{2}[A-Z]{2}[0-9]{3}[A-Z]?$`; semester 1–8; credits 1–6.
- **units**: unit_no 1–5, UNIQUE per course. **topics**: name UNIQUE per unit.
- **faculty_courses**: composite PK; trigger allows only role = faculty.
- **repeat_groups**: course_id, label (text of the first appearance), created_by (NULL = created by the detector).
- **bookmarks**: composite PK; trigger allows only questions of published papers.
- **detection_settings**: single row (CHECK id = 1); CHECK 0 < review_threshold < auto_threshold ≤ 1.
- **audit_log**: written only by triggers; JSONB snapshots of old/new rows.

9. SQL implementation

All SQL is in `db/` and runs in order with `npm run db:setup`.

9.1 DDL (`01_schema.sql`)

- **DROP** of every view, table and type at the start (script is re-runnable).
- **CREATE TYPE** for four ENUMs, **CREATE TABLE** for 13 tables, **CREATE EXTENSION** vector.
- **ALTER TABLE** adds the foreign keys of `students` and `questions` after all tables exist, adds the `duration_min` column and its CHECK.
- Constraints used: PRIMARY KEY (single and composite), FOREIGN KEY with CASCADE / SET NULL / RESTRICT, UNIQUE (single and composite), NOT NULL, DEFAULT, CHECK (ranges, regex patterns, cross-column), GENERATED column.

9.2 Indexes (`02_indexes.sql`)

Index	Type	Used by
idx_questions_paper, idx_questions_topic, idx_units_course, ...	B-tree	joins on foreign keys
idx_papers_course_year (course_id, exam_year DESC, exam_type)	composite B-tree	course pages, newest papers first
idx_questions_grouped WHERE group_id IS NOT NULL	partial	repeat-group lookups
idx_candidates_pending WHERE status = 'pending'	partial	review queue
idx_questions_tsv	GIN	full-text search
idx_questions_embedding (vector_cosine_ops)	HNSW	nearest-neighbour search, repeat detector

9.3 Functions and stored procedures (`03_functions_triggers.sql`)

Name	Kind	Purpose
fn_question_course, fn_similarity, fn_can_manage_course	SQL functions	helpers (course of a question, cosine similarity of two questions, permission check)
fn_detect_repeats(question_id)	PL/pgSQL function	**the repeat detector**: nearest 8 questions of the same course in other papers; ≥ auto threshold → group, between thresholds → review queue
fn_link_questions(a, b, actor)	PL/pgSQL function	put two questions in one group: create, join or merge groups; settle redundant suggestions
sp_review_candidate(low, high, decision, reviewer)	procedure	faculty decision with row lock and permission check
sp_ungroup_question(question, actor)	procedure	remove from group, remember "not a repeat"
sp_merge_groups(keep, drop, actor)	procedure	merge two groups of one course
sp_publish_paper(paper, actor)	procedure	publish only if it has questions and all are embedded

9.4 Triggers

Trigger	When	Rule
trg_question_integrity	BEFORE INSERT/UPDATE on questions	topic and group must belong to the paper's course; published paper is locked; editing text clears the stale embedding and group
trg_question_delete_guard	BEFORE DELETE on questions	cannot delete questions of a published paper

Trigger	When	Rule
trg_detect_repeats	AFTER INSERT / UPDATE OF embedding	runs `fn_detect_repeats`
trg_prune_groups	AFTER UPDATE OF group_id / DELETE	deletes groups left with fewer than 2 questions
trg_paper_guard	BEFORE UPDATE on exam_papers	published papers cannot be unpublished or moved
trg_faculty_role_check	BEFORE INSERT/UPDATE on faculty_courses	only faculty can be assigned
trg_bookmark_check	BEFORE INSERT on bookmarks	only questions of published papers
trg_audit_*	AFTER INSERT/UPDATE/DELETE	write JSONB snapshots to audit_log (`05_audit.sql`)

9.5 Views (`04_views.sql`)

View	Concepts	Used on
v_question_detail	5-table JOIN, derived group size	everywhere a question is shown
v_repeat_groups	GROUP BY, HAVING, COUNT(DISTINCT), string_agg ... ORDER BY	group statistics
v_hot_questions	CTE, window function RANK() OVER (PARTITION BY course)	"Most repeated" ranking
v_unit_weightage	LEFT JOIN of a join, window SUM over a SUM	unit weightage bars
v_topic_frequency	LEFT JOIN, aggregates	topic table
v_paper_summary	correlated EXISTS inside an aggregate FILTER, row comparison `(year, type) < (year, type)`	share of each paper repeated from earlier papers

9.6 Transactions, concurrency and locking

- Every write from the app runs in `withTransaction()` (`src/lib/db.ts`): `BEGIN`, the statements, `COMMIT`, or `ROLLBACK` on any error. Example: student registration inserts into `users` and `students` atomically, so an invalid SRN never leaves a half-created account.
- Adding a question is one transaction: the INSERT fires the detector, which may create a group, update other questions and insert suggestions. Either all of it happens or none.
- Procedures lock the rows they change with `SELECT … FOR UPDATE`. Two faculty members reviewing the same pair at the same moment cannot both succeed: the second waits for the lock, then sees the pair is no longer pending ("already reviewed"). Test 30 demonstrates the blocking with two connections.
- The acting user is passed to the database with `set_config('app.user_id', …, true)`, a setting local to the transaction, which the audit trigger reads.

10. Vector database and semantic search

Question in the handout	Answer
What data is converted into embeddings?	The text of every question (`questions.body`), and the search text a user types.

Question in the handout	Answer
Which embedding model?	`sentence-transformers/all-MiniLM-L6-v2` (int8-quantised ONNX), run locally in Node.js through `@huggingface/transformers`. 384 dimensions, mean pooling, L2-normalised. No external API or key.
How are vectors stored?	In the same row as the question: `questions.embedding vector(384)` (pgvector), indexed with **HNSW** (`vector_cosine_ops`).
Similarity metric	**Cosine** distance `<=>`; similarity = 1 − distance (vectors are normalised, so this equals the dot product).
How are relevant results retrieved?	`ORDER BY embedding <=> $query LIMIT k`, combined with relational filters (same course, other paper, published only) in the same SQL statement.

Threshold calibration. We embedded every seed question and compared all pairs:

Pair type	Median similarity	Max
Rewordings of the same question	0.80	0.94
Different questions, same topic	0.38	0.65
Different questions, same unit	0.24	0.73

So the defaults are **auto-group ≥ 0.80** (no different-question pair in the data reaches that) and **review 0.60–0.80** (catches most remaining rewordings, plus some near-misses for a human to reject). Admins can tune both on the Detection page; a CHECK constraint keeps them consistent.

Measured result on the seed data (`docs/detection-evaluation.md`, generated by the seed script, which knows the true families):

Measure	Value
Auto-grouped pairs that really are repeats (precision)	**100%**
True repeats found either automatically or via the review queue (recall)	**99.1%**
Pairs sent to review that really are repeats	61.8% (the rest are correctly rejected by faculty)

11. Important queries

Repeat detector core (inside `fn_detect_repeats`): nearest questions of the same course in other papers.

```
SELECT q2.question_id, q2.group_id,
       ROUND((1 - (q2.embedding <=> v))::numeric, 4) AS sim
FROM questions q2
JOIN exam_papers p2 ON p2.paper_id = q2.paper_id
WHERE p2.course_id = my_course
  AND q2.paper_id <> my_paper
  AND q2.embedding IS NOT NULL
ORDER BY q2.embedding <=> v
LIMIT 8;
```

Most likely questions (view `v_hot_questions`, CTE + window function):

```
WITH latest AS (SELECT course_id, MAX(exam_year) AS latest_year
                FROM exam_papers WHERE status = 'published' GROUP BY course_id)
SELECT rg.*,
       RANK() OVER (PARTITION BY rg.course_id
                    ORDER BY rg.distinct_years
                             + CASE WHEN l.latest_year - rg.last_year <= 1 THEN 1 ELSE 0 END
DESC,
                             rg.appearances DESC) AS hot_rank
FROM v_repeat_groups rg JOIN latest l ON l.course_id = rg.course_id;
```

How much of each paper was repeated (correlated subquery in `v_paper_summary`):

```
SUM(q.marks) FILTER (WHERE EXISTS (
    SELECT 1 FROM questions q2 JOIN exam_papers p2 ON p2.paper_id = q2.paper_id
    WHERE q2.group_id = q.group_id AND p2.status = 'published'
      AND (p2.exam_year, p2.exam_type) < (p.exam_year, p.exam_type)))
```

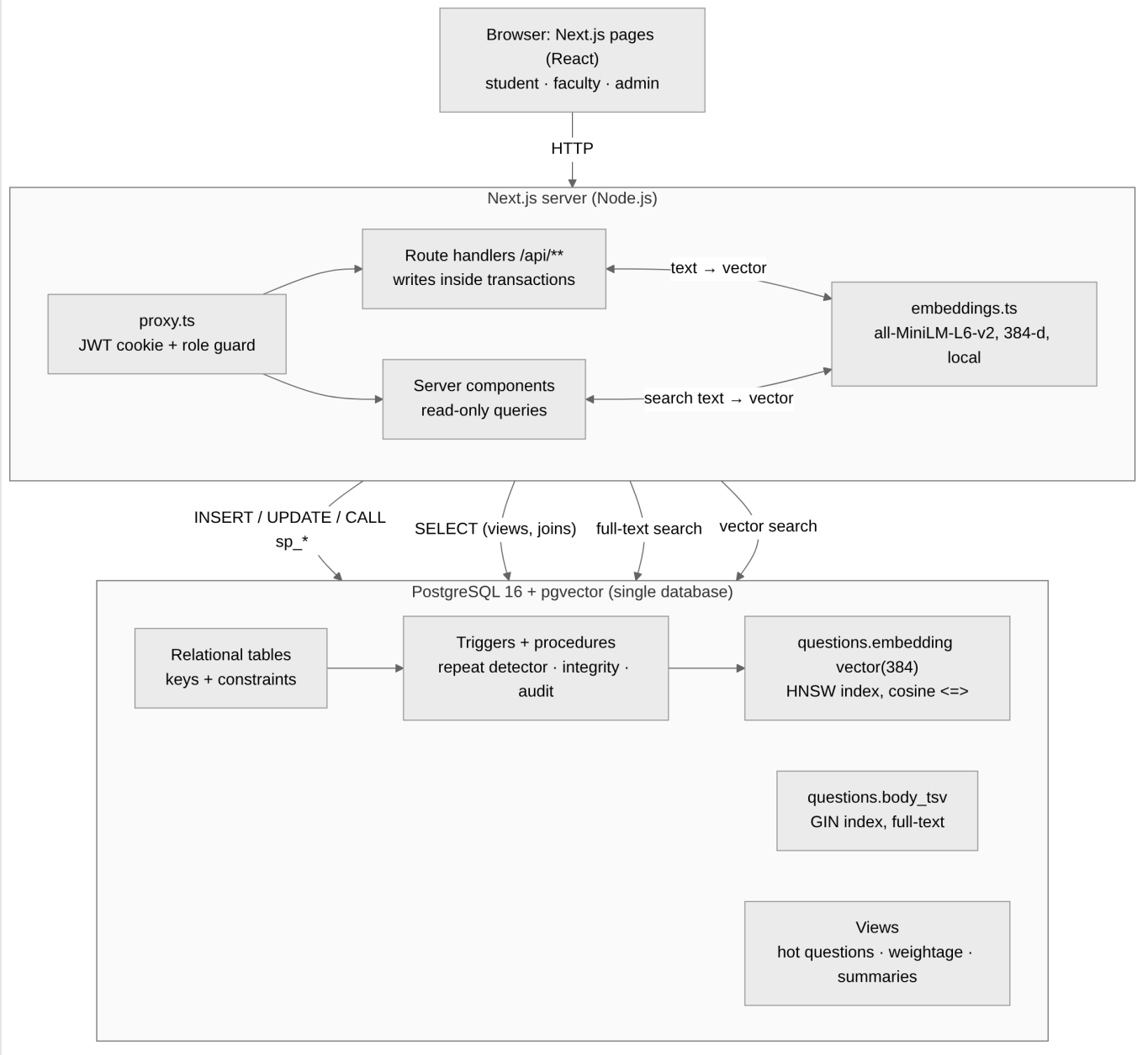
Keyword vs meaning search (`/bank/search`):

```
-- full-text
SELECT … , ts_rank(q.body_tsv, websearch_to_tsquery('english', $1)) AS score
FROM … WHERE q.body_tsv @@ websearch_to_tsquery('english', $1) ORDER BY score DESC;
-- vector
SELECT … , 1 - (q.embedding <=> $1::vector) AS score
FROM … ORDER BY q.embedding <=> $1::vector LIMIT 15;
```

Relational algebra for two of the SQL Lab queries: - Topics never asked in 2024 or 2025: $\pi_{name}(\text{topics}) - \pi_{name}(\text{topics} \bowtie \text{questions} \bowtie \sigma_{exam_year \in \{2024,\ 2025\}}(\text{exam_papers}))$ - Topics asked in at least 4 years: $\sigma_{years \geq 4}(\ {}_{topic_id}\mathcal{G}_{count(distinct\ exam_year) \rightarrow years}(\text{questions} \bowtie \text{exam_papers})\ )$

The admin **SQL Lab** runs 15 such queries live: multi-table JOIN; aggregate functions; GROUP BY + HAVING; correlated subquery; NOT EXISTS; EXCEPT / INTERSECT / UNION ALL; window functions RANK and LAG; CTE; full-text search; vector self-join with LATERAL; a view; stored functions; two EXPLAIN plans (query optimisation: on a small table the planner prefers a sequential scan; with sequential scans discouraged it switches to a Bitmap Index Scan on the GIN index); and the audit summary.

12. System architecture



- **Frontend**: Next.js 16 App Router pages (React 19, Tailwind CSS 4).
- **Backend/API**: Next.js server components (reads) and route handlers under `src/app/api/**` (writes), using a `pg` connection pool. The database is never reached from the browser.
- **Database**: one PostgreSQL 16 database with the pgvector extension.
- **Configuration**: `DATABASE_URL` and `AUTH_SECRET` come only from `.env` (git-ignored); `.env.example` documents them.

Flow of "faculty adds a question": form → `POST /api/faculty/papers/:id/questions` → role check and course-assignment check → `embedText(body)` → transaction: `INSERT INTO questions (…, embedding)` → `trg_question_integrity` validates → `trg_detect_repeats` → `fn_detect_repeats` (HNSW search) → `fn_link_questions` or `INSERT INTO repeat_candidates` → audit triggers → `COMMIT` → the page shows "Repeat detected", "sent for review" or "new question".

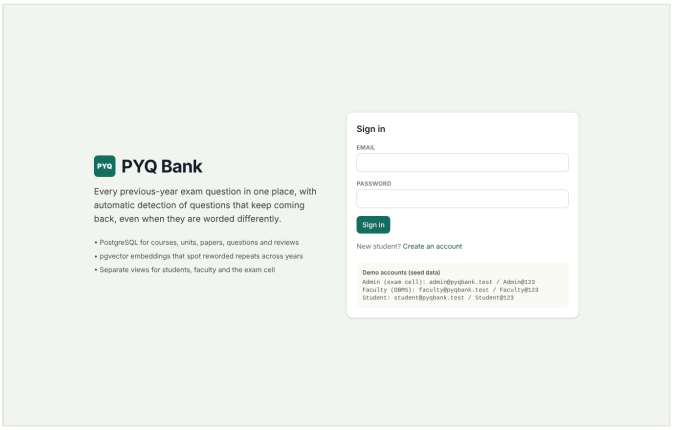
13. Authentication and authorisation

- **Authentication**: passwords hashed with bcrypt (10 rounds). On login the server verifies the hash and sets a signed JWT (HS256, 8 hours) in an `httpOnly`, `SameSite=Lax` cookie. Logging in with an unknown email or wrong password gives the same message (no account enumeration). Disabled accounts cannot log in.
- **Three roles**: admin, faculty, student.
- **Authorisation in three layers**:
- `src/proxy.ts` checks the token and role before any `/admin`, `/faculty`, `/student`, `/bank` page or API route.
- Every API route checks the role again (`requireApi`) and, for course-specific actions, that the faculty member is assigned to the course.
- The **database** checks it too: `sp_review_candidate`, `sp_ungroup_question`, `sp_merge_groups` and `sp_publish_paper` call `fn_can_manage_course` and refuse otherwise (test 24).

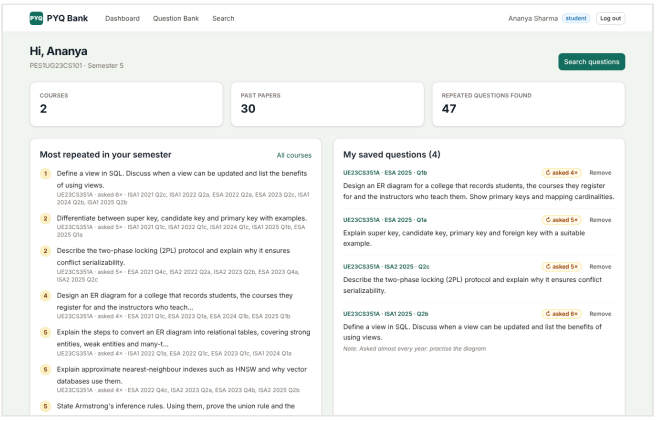
14. Data validation and integrity

Rule	Enforced by
Valid email, SRN, course code, question label formats	CHECK with regular expressions
Marks, years, semesters, credits, thresholds in range	CHECK
No duplicate account, SRN, course, paper, question number, topic	UNIQUE (single and composite)
Only known roles, exam types, statuses	ENUM types
Every reference points to an existing row	FOREIGN KEY, with CASCADE / SET NULL / RESTRICT chosen per relationship
A course with papers, or a topic used by questions, cannot be deleted	ON DELETE RESTRICT
A question's topic belongs to the paper's course	trigger
Published papers are read-only and cannot be unpublished	triggers
A pair is reviewed once; review fields are consistent with status	row lock in procedure + CHECK
Only faculty are assigned to courses; only published questions are bookmarked	triggers
Multi-step operations are all-or-nothing	transactions

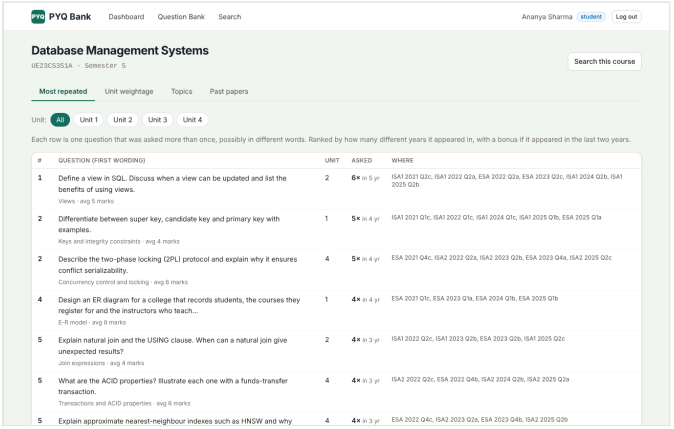
15. Application screenshots



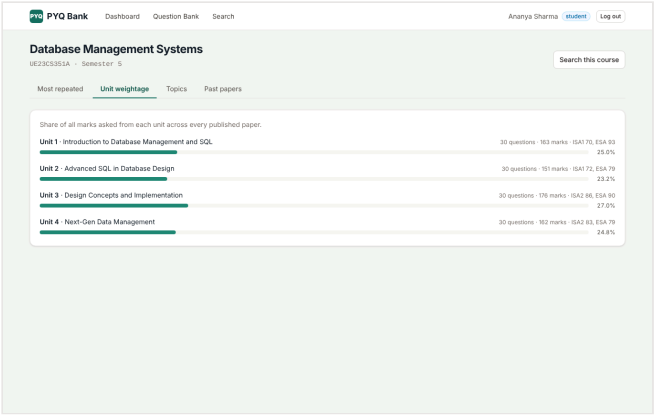
Login



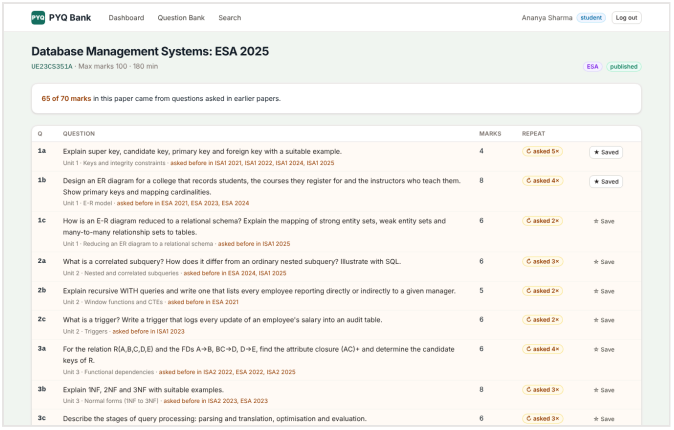
Student dashboard



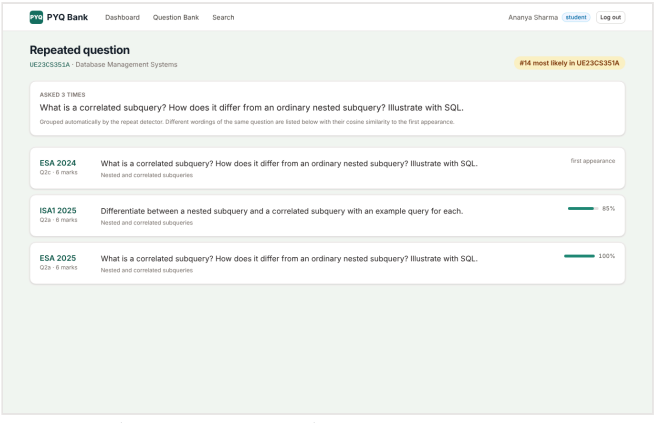
Most repeated questions



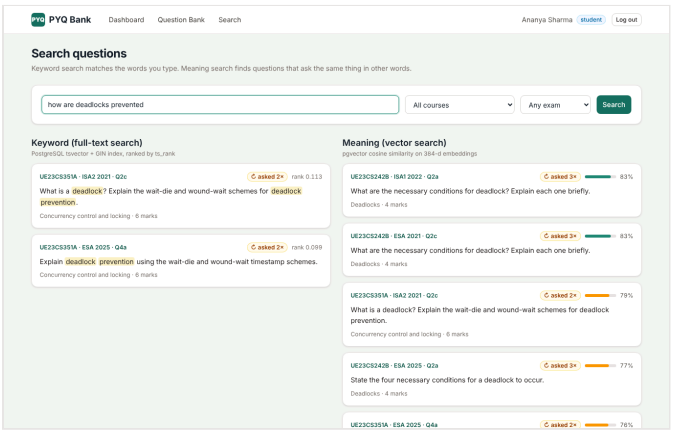
Unit weightage



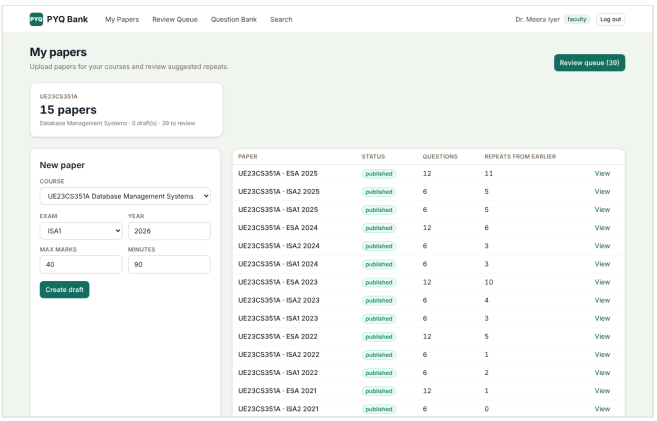
Paper with repeat markers



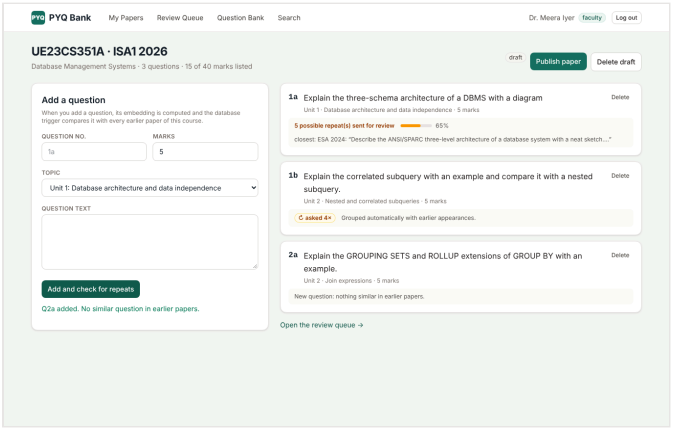
One question, several wordings



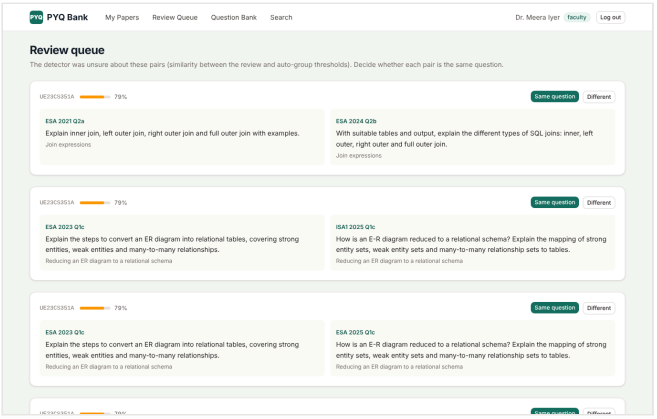
Keyword vs meaning search



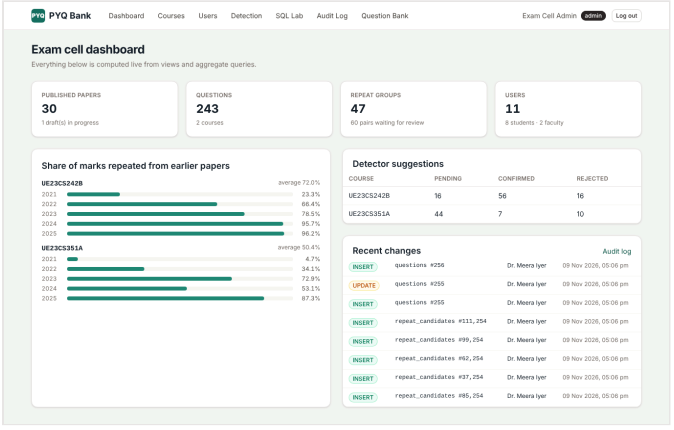
Faculty dashboard



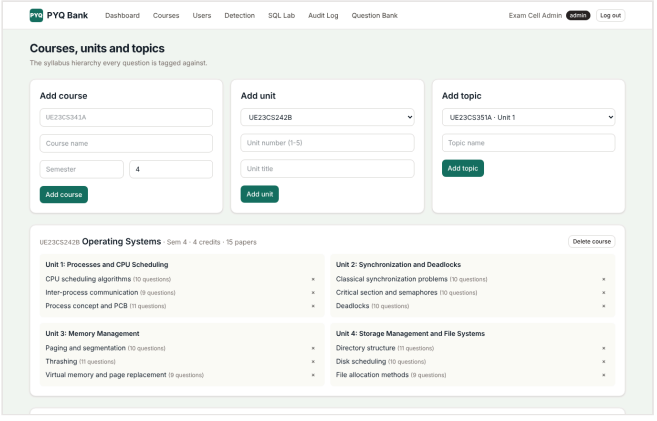
Live repeat detection on upload



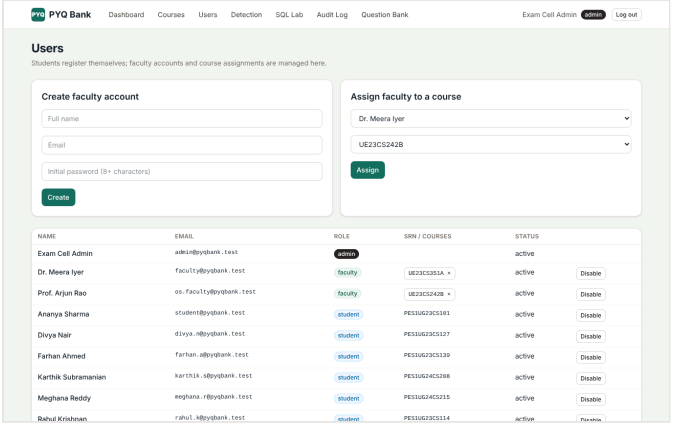
Review queue



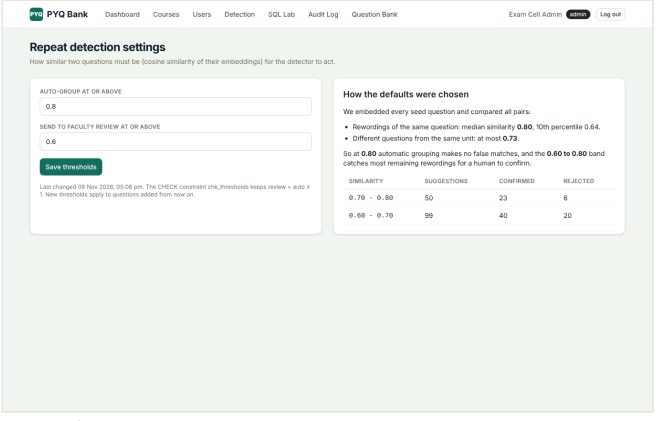
Admin dashboard



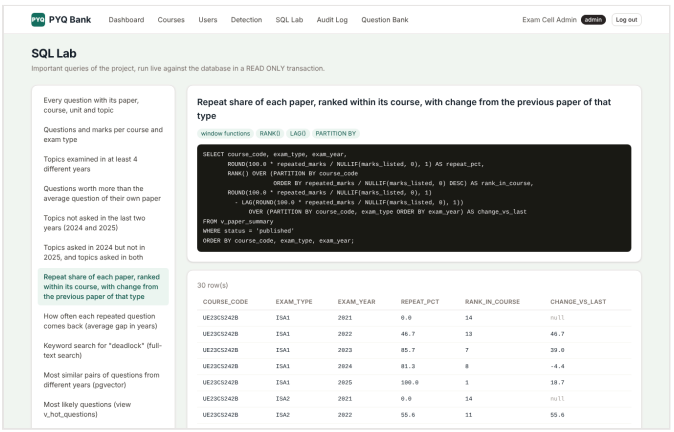
Courses, units, topics



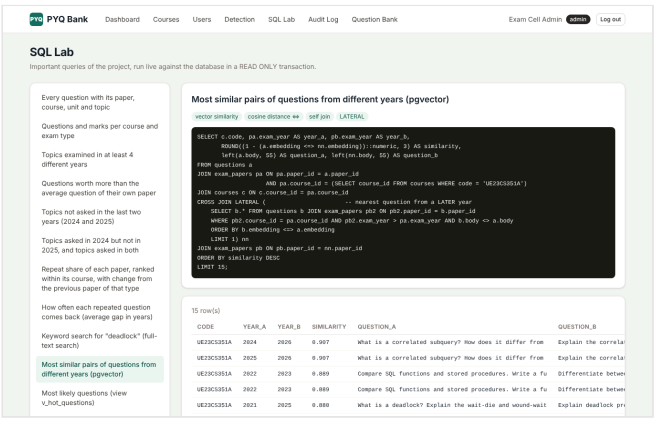
Users and assignments



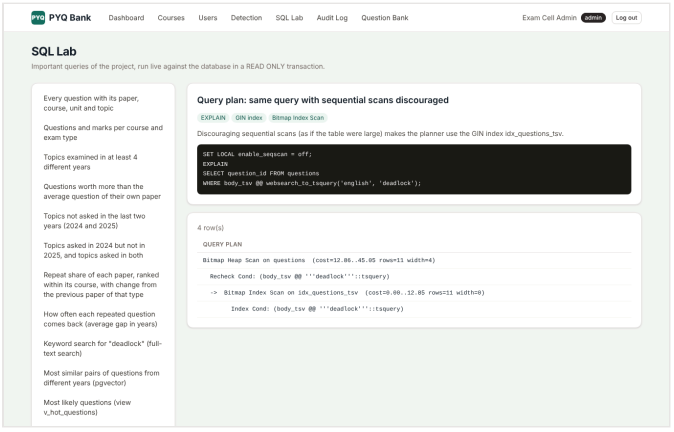
Detection thresholds



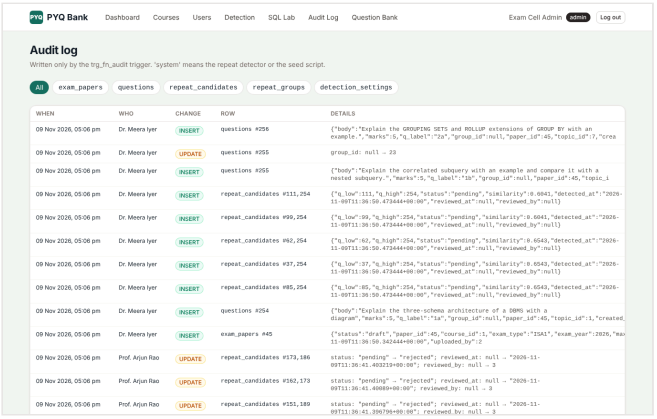
SQL Lab: window functions



SQL Lab: vector similarity



SQL Lab: query plan using the GIN index



Audit log

16. Testing

Database test suite (`npm run test:db`, results in `docs/test-results.md`): **34 test cases, all passing** on PostgreSQL 16 + pgvector. Every test runs in a transaction that is rolled back.

Area	Tests
Constraints	SRN format, email format, duplicate email, duplicate paper, marks range, question label, threshold order, ENUM, FK RESTRICT
Triggers	topic from another course, edit/delete in published paper, unpublish, stale embedding cleared, faculty role, bookmark of draft
Repeat detector	exact copy auto-grouped; similarity 0.70 sent to review; unrelated not flagged; other course never matched; higher thresholds stop grouping
Procedures	confirm links two questions; no double review; faculty of another course refused; ungroup remembered; merge groups; publish empty or un-embedded paper refused
Transactions	failure rolls back earlier statements; second session blocked by `FOR UPDATE` lock
Views and search	view totals equal base-table totals; unit percentages sum to 100; full-text stem match; nearest neighbour of a vector is itself

Detector evaluation: see section 10 (100% precision of automatic grouping, 99.1% recall including the review queue).

End-to-end: every page of all three roles was exercised in a headless browser (Playwright): login, role redirects (a student opening `/admin` is sent back to `/student`), bookmarking, creating a paper, adding

three questions (one grouped automatically, one sent to review, one new), confirming a pair, publishing, all 15 SQL Lab queries, and saving an invalid threshold (rejected by the CHECK constraint with a readable message).

17. Individual contributions

Member	Database work	Application work
Swaroop Venkateshwar (PES1UG24CS923)	Schema and DDL (`01_schema.sql`), indexes (`02_indexes.sql`), functions, procedures and triggers including the repeat detector (`03_functions_triggers.sql`), seed data and detector calibration	DB access layer and transactions (`db.ts`), embeddings, setup/seed/local-DB scripts, base layout and UI components, student module (dashboard, bookmarks), question-bank pages (course, paper, repeat group) and search
Shreesh S A (PES1UG24CS441)	Views (`04_views.sql`), audit trail (`05_audit.sql`), database test suite (`test-db.mjs`), SQL Lab queries	Authentication and role-based access (JWT sessions, proxy, login/register), faculty module (paper upload with live detection, review queue, ungroup), admin module (dashboard, courses, users, settings, SQL Lab, audit log), README and report

The Git history shows each member's commits.

18. Conclusion

PYQ Bank shows a clear link between DBMS concepts and a real problem. The relational part (normalised to BCNF, with constraints, triggers, procedures and views) keeps exam data consistent and answers the analytical questions students care about: what is asked most, from which unit, and how often. The vector part answers the one question relational equality cannot: *is this the same question in different words?* Because the vectors live in PostgreSQL itself, the detector runs as a trigger inside the same transaction as the insert, combining nearest-neighbour search with relational filters in one query. On the seed data it groups repeats with no false matches and finds 99% of them when the review queue is included.

Limitations and future work - The small embedding model misses some rewordings (e.g. "Explain 1NF, 2NF, 3NF" vs "describe the first, second and third normal forms"). Topic-level statistics still count these, and a larger model would improve recall. - Questions are entered one at a time. A PDF import with text extraction would speed up uploading. - Changing the thresholds applies to new questions only; a "re-scan" procedure could re-run detection over existing ones. - The demo data is illustrative (realistic questions, generated papers); real papers can be added through the faculty upload page.

Appendix: how to run

See `README.md`. In short: `npm install`, set `.env`, `npm run db:setup`, `npm run db:seed`, `npm run dev`.